

Multimodal Deep Learning

Solutions to Exercise Sheet 2

Date: 13th of May, 2025

Exercise 1: Numerical Features: Normalization vs. Standardization

A one-dimensional sensor produces readings

$$\mathbf{x} = [2.7, 3.1, 9.4, 6.5, 4.2, 3.8].$$

- Apply **min-max normalization** to map the data into $[0, 1]$.
- Apply **z-score standardization**.
- When would you prefer Min-Max scaling over z-scaling, and vice-versa?
- Binning / Discretization:** Why convert a continuous feature (e.g. “age”) into bins (“young”, “middle-aged”, “senior”)? Give one benefit and one drawback.

Solution

```
(a) import numpy as np
2 x = np.array([2.7, 3.1, 9.4, 6.5, 4.2, 3.8])
3
4 # Min-Max
5 x_minmax = (x - x.min()) / (x.max() - x.min())
```

$$\mathbf{x}_{\text{min-max}} = [0, 0.059701, 1, 0.567164, 0.223881, 0.164179].$$

```
(b) import numpy as np
2 x = np.array([2.7, 3.1, 9.4, 6.5, 4.2, 3.8])
3
4 # z-score
5 x_z = (x - x.mean()) / x.std()
```

$$\mathbf{z} = [-0.965642, -0.793972, 1.909824, 0.665220, -0.321881, -0.493550].$$

- **Min-Max scaling** is preferred when the true range is known or when algorithms require inputs in a bounded interval. It is, however, sensitive to outliers because they stretch the range.
 - **z-score scaling** is preferred when features are (approximately) Gaussian or when zero mean and unit variance benefit the algorithm.
- **Benefit:** improved interpretability and robustness (small measurement errors within a bin do not change the category).
 - **Drawback:** information loss; values like 44 and 59 years become indistinguishable, which can hurt predictive performance.

Exercise 2: Categorical Features

You have a categorical attribute `Color` with values

[Red, Blue, Green, Blue].

- Show the One-Hot encoded matrix.
- Show the Label-Encoding (integers assigned alphabetically from 0).
- What can go wrong if you feed the labels from (b) directly into a linear or distance-based model?

Solution

(a)

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix},$$

(b) Assigning integer labels alphabetically from 0 gives

Blue $\mapsto$ 0, Green $\mapsto$ 1, Red $\mapsto$ 2.

The encoded vector is therefore

$$\mathbf{z} = [2, 0, 1, 0]^T.$$

(c) **Why direct use of (b) can be harmful**

Label-encoded values impose an *artificial order and metric* on nominal categories. A linear model would interpret “Red (2)” as larger than “Green (1)” and “Blue (0)”, and a distance-based algorithm would treat $|2 - 1| = 1$ as meaning that Red is “closer” to Green than to Blue. Because the numeric gaps carry no real semantic meaning for colors, the model could learn spurious relationships, leading to biased coefficients, distorted decision boundaries, and degraded predictive performance. One-hot encoding avoids this by keeping the categories equidistant and non-ordinal.

Exercise 3: Text Features

Byte-pair encoding (BPE) corpus

["low", "lower", "lowest", "newest"]

Perform 4 BPE merges. Show the merge pairs and the updated vocabulary after each step.

Solution

Corpus: "low", "lower", "lowest", "newest". 4 merges. Tie-break: First Encountered.

Initial State: Vocab V_0 : {e, l, n, o, r, s, t, w}.

Tokenizations: "low": l o w "lower": l o w e r "lowest": l o w e s t "newest": n e w e s t

Frequencies: ('l', 'o'): 3, ('o', 'w'): 3, ('w', 'e'): 3, ('e', 's'): 2, ('s', 't'): 2, ('e', 'r'): 1, ('n', 'e'): 1, ('e', 'w'): 1

Merge 1: Pair ('l', 'o') → **lo** (Freq: 3). Add **lo** to vocab.

Tokenizations: "low": **lo** w "lower": **lo** w e r "lowest": **lo** w e s t "newest": n e w e s t
 Frequencies: ('lo', 'w'): 3, ('w', 'e'): 3, ('e', 's'): 2, ('s', 't'): 2, ('e', 'r'): 1, ('n', 'e'): 1, ('e', 'w'): 1

Merge 2: Pair ('lo', 'w') → **low** (Freq: 3). Add **low**.

Tokenizations: "low": **low** "lower": **low** e r "lowest": **low** e s t "newest": n e w e s t
 Frequencies: ('low', 'e'): 2, ('e', 's'): 2, ('s', 't'): 2, ('e', 'r'): 1, ('n', 'e'): 1, ('e', 'w'): 1, ('w', 'e'): 1

Merge 3: Pair ('low', 'e') → **lowe** (Freq: 2). Add **lowe**.

Tokenizations: "low": **low** "lower": **lowe** r "lowest": **lowe** s t "newest": n e w e s t
 Frequencies: ('s', 't'): 2, ('lowe', 'r'): 1, ('lowe', 's'): 1, ('n', 'e'): 1, ('e', 'w'): 1, ('w', 'e'): 1, ('e', 's'): 1

Merge 4: Pair ('s', 't') → **st** (Freq: 2). Add **st**.

Final Tokenizations: "low": **low** "lower": **lowe** r "lowest": **lowe** s t "newest": n e w e s t

Final Frequencies: ('lowe', 'r'): 1, ('lowe', 'st'): 1, ('n', 'e'): 1, ('e', 'w'): 1, ('w', 'e'): 1, ('e', 'st'): 1

Notes: • Atomic chars **l, o, s, t** remain in V_4 but unused individually here. • BPE targets efficiency, not max compression; unused symbols could be pruned.

Exercise 4: Johnson–Lindenstrauss Lemma Bounds

The *Johnson–Lindenstrauss lemma* states that any set of N points in a high-dimensional Euclidean space can be embedded into $\mathbb{R}^k$ with only a small distortion of pairwise distances. Formally, for any $0 < \epsilon < 1$ and every set of N points in $\mathbb{R}^D$, there exists a linear map

$$f: \mathbb{R}^D \longrightarrow \mathbb{R}^k$$

such that for all points u, v in the set

$$(1 - \epsilon) \|u - v\|_2^2 \leq \|f(u) - f(v)\|_2^2 \leq (1 + \epsilon) \|u - v\|_2^2,$$

provided the target dimension k satisfies

$$k \geq \mathcal{O}\left(\frac{\log N}{\epsilon^2}\right).$$

A more concrete lower bound often cited is

$$k \geq \frac{C \log N}{\epsilon^2}$$

for some universal constant C . For simplicity in this exercise, we use the bound $C = 8$.

- $N = 10^6$, $\epsilon = 0.1$: compute the minimum k .
- With $k = 500$ and $N = 10^6$, what is the *best guaranteed* distortion ϵ ?
- Random projections vs. PCA: what do they each optimise or preserve?

Solution

- Using the explicit bound with $C = 8$,

$$k \geq \frac{8 \log N}{\epsilon^2} = \frac{8 \log(10^6)}{0.1^2} \approx 11052.$$

Hence a safe choice is

$$k_{\min} = 11\,053.$$

(b) Rearranging the same bound to solve for ϵ gives

$$\epsilon \geq \sqrt{\frac{C \log N}{k}} = \sqrt{\frac{8 \log(10^6)}{500}} \approx 0.47.$$

Thus with $k = 500$ and $N = 10^6$, the best worst-case *guaranteed* distortion is

$$\epsilon_{\min} \approx 0.47 \quad (\text{or } 47\%).$$

(c) Both have in common linear transformations.

- **Random projections** (e.g. using a Johnson–Lindenstrauss transform) are *data-independent*. They aim to *preserve all pairwise Euclidean distances* up to the factor $(1 \pm \epsilon)$ with high probability.
- **Principal Component Analysis (PCA)** is *data-dependent*. It chooses orthogonal directions that maximise the retained *total variance* (equivalently, minimise mean-squared reconstruction error of the original points). PCA makes no uniform guarantee on individual pairwise distances; distances along discarded components can shrink substantially.

Exercise 5: Sparse vs. Dense Distance Functions

- (a) **Sparse binary vectors:** User $A = [0, 0, 0, 1, 0, \dots]$ and user $B = [1, 1, 0, 0, 0, \dots]$ encode rare game achievements. Explain why Euclidean distance is unsuitable for very high dimensional sparse vectors, and propose a better metric.
- (b) **Unit-normalised dense vectors:**

- Show that for unit-length vectors u, v

$$\|u - v\|_2^2 = 2 - 2 \cos \theta.$$

- Interpret the relationship between Euclidean distance and cosine similarity.
- Two d -dimensional, L^2 -normalised embeddings have cosine similarity 0.9.
Express $\|v_1 - v_2\|_2$ in terms of θ and d .

- (c) **Dense count vectors of different length:** For raw word-count vectors of unequal document length, argue why cosine similarity is preferable to Euclidean distance.

Solution

- (a) Let $A, B \in \{0, 1\}^d$ encode rare achievements.

- *Why Euclidean distance is ill-suited.* For binary incidence vectors $A, B \in \{0, 1\}^N$ the Euclidean distance is

$$d_E(A, B) = \sqrt{\sum_{i=1}^N (a_i - b_i)^2} = \sqrt{|A \setminus B| + |B \setminus A|},$$

which *ignores all 0-coordinates*. When N is very large and $|A|, |B| \ll N$, almost every coordinate is 0 in both vectors, so d_E is essentially the same for any pair with the same numbers of rare achievements, even if they share none. Thus Euclidean distance fails to capture *overlap* in highly-sparse settings.

- *Better metric.*

Cosine similarity: The dot product focuses only on dimensions where both vectors are 1. Shared 0s do not contribute to the dot product or the magnitudes. It measures orientation similarity, effectively normalizing for the number of achievements each user has.

(b)

$$\|x - y\|_2^2 = (x - y)^\top (x - y) = x^\top x - 2x^\top y + y^\top y = 2 - 2x^\top y = 2 - 2\cos\angle(x, y)$$

Interpretation. The map $s \mapsto d(u, v) = \sqrt{2 - 2s}$ is monotone decreasing on $s = \cos\theta \in [-1, 1]$, so *ranking* by Euclidean distance or by cosine similarity is equivalent after L^2 normalisation. Geometrically, distance measures the chord length of two points on the unit hypersphere, while cosine similarity measures the central angle.

Embeddings with cosine similarity 0.9. Let $\cos\theta = 0.9$. As above,

$$\|\mathbf{v}_1 - \mathbf{v}_2\|_2 = \sqrt{2 - 2\cos\theta} = \sqrt{0.2} = 2\sin(\frac{\theta}{2}) \approx 0.447.$$

The expression is *dimension-independent*; d plays no role once vectors are L^2 -normalised.

(c) **Dense count vectors of different length**

Let $\mathbf{x} \in \mathbb{R}^d$ be the word-count vector of a short note and $\mathbf{y} = k\mathbf{x}$ ($k > 1$) the counts for a longer version of the same text.

Euclidean distance: $\|\mathbf{x} - \mathbf{y}\| = (k - 1)\|\mathbf{x}\| \rightarrow$ grows with document length k .

Cosine similarity: $\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{\mathbf{x} \cdot k\mathbf{x}}{\|\mathbf{x}\| k\|\mathbf{x}\|} = 1 \rightarrow$ invariant to length.

Scaling a count vector by k (adding or removing repetitions) changes its magnitude but not its direction. Euclidean distance therefore over-penalises longer documents, whereas cosine compares only the relative word frequencies, making it the preferred measure for raw count vectors of different lengths.

Exercise 6: PCA via SVD

Given the mean-centred matrix

$$\mathbf{X} = \begin{bmatrix} -1 & 2 & 0 \\ -1 & -3 & 1 \\ 2 & 1 & -1 \end{bmatrix},$$

perform the singular-value decomposition $\mathbf{X} = U\Sigma V^\top$. (Feel free to use a linear-algebra library such as ‘numpy.linalg.svd’.)

- Give U , Σ , and $V^\top$.
- Using the first principal component, form the rank-1 reconstruction $\mathbf{X}_1$, compute the reconstruction error $\|\mathbf{X} - \mathbf{X}_1\|_F$ using the Frobenius norm.
- For a general data set, how can we choose k so that the reconstruction retains at least α fraction of the total variance?
- Relate principal components to *intrinsic dimension*.

Solution**1. SVD of the centred data**

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

- Columns of $\mathbf{V}$ are the *principal directions / axes*.
- Columns of $\mathbf{U}\mathbf{\Sigma}$ are the *principal components (scores)*.
- Singular values relate to covariance-matrix eigenvalues by

$$\lambda_i = \frac{\sigma_i^2}{n-1},$$

so λ_i equals the variance explained by PC i .

2. Dimensionality reduction to $k < p$

$$\mathbf{U}_k = \mathbf{U}(:, 1:k), \quad \mathbf{\Sigma}_k = \mathbf{\Sigma}(1:k, 1:k).$$

Scores (first k PCs) : $\mathbf{Z} = \mathbf{U}_k \mathbf{\Sigma}_k \in \mathbb{R}^{n \times k}$.

Rank- k reconstruction : $\mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$

Exercise

Data (already mean-centred)

$$\mathbf{X} = \begin{bmatrix} -1 & 2 & 0 \\ -1 & -3 & 1 \\ 2 & 1 & -1 \end{bmatrix}.$$

1. Compute the SVD

Using `numpy.linalg.svd`:

$$\mathbf{U} = \begin{bmatrix} -0.351843 & 0.736799 & 0.577350 \\ 0.814008 & -0.063694 & 0.577350 \\ -0.462165 & -0.673105 & 0.577350 \end{bmatrix}, \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \sigma_3),$$

$$\sigma_1 = \sqrt{11 + \sqrt{31}} \approx 4.0704, \quad \sigma_2 = \sqrt{11 - \sqrt{31}} \approx 2.3307, \quad \sigma_3 = 0,$$

$$\mathbf{V}^\top = \begin{bmatrix} -0.340633 & -0.866393 & 0.365148 \\ -0.886380 & 0.425438 & 0.182574 \\ 0.313529 & 0.261469 & 0.912871 \end{bmatrix}.$$

(Check: $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \mathbf{X}$.)

2. Rank-1 reconstruction $\mathbf{X}_1$ and error

(a) **Step 1:** reconstruction formula for $k = 1$.

$\mathbf{X}_1 = \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^\top = \mathbf{u}_1 \sigma_1 \mathbf{v}_1^\top$, where $\mathbf{u}_1$ is the first column of $\mathbf{U}$, σ_1 is the first singular value, and $\mathbf{v}_1^\top$ is the first row of $\mathbf{V}^\top$.

(b) **Step 2:** Extract the required components from Part (a).

- $\sigma_1 = \sqrt{11 + \sqrt{31}} \approx 4.0704$

- $\mathbf{u}_1 = [-0.351843, 0.814008, -0.462165]^\top$
- $\mathbf{v}_1^\top = [-0.340633, -0.866393, 0.365148]$

(c) **Step 3:** Calculate $\mathbf{X}_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top$.

$$\mathbf{X}_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top \approx 4.0704 \begin{bmatrix} -0.351843 \\ 0.814008 \\ -0.462165 \end{bmatrix} \begin{bmatrix} -0.340633 & -0.866393 & 0.365148 \end{bmatrix}$$

Calculating the outer product gives:

$$\mathbf{X}_1 \approx \begin{bmatrix} 0.48783 & 1.26943 & -0.52294 \\ -1.12862 & -2.93689 & 1.20984 \\ 0.64079 & 1.66746 & -0.68691 \end{bmatrix}$$

(d) **Step 4:**

The Frobenius norm of the reconstruction error using the first k components is the square root of the sum of squares of the discarded singular values:

$$\|\mathbf{X} - \mathbf{X}_k\|_F = \sqrt{\sum_{j=k+1}^r \sigma_j^2}$$

For $k = 1$, the discarded singular values are σ_2 and σ_3 :

$$\|\mathbf{X} - \mathbf{X}_1\|_F = \sqrt{\sigma_2^2 + \sigma_3^2}$$

$$\sigma_2 = \sqrt{11 - \sqrt{31}} \approx 2.3307 \text{ and } \sigma_3 = 0.$$

$$\|\mathbf{X} - \mathbf{X}_1\|_F = \sqrt{(\sqrt{11 - \sqrt{31}})^2 + 0^2} = \sqrt{11 - \sqrt{31}} = \sigma_2$$

The reconstruction error is:

$$\|\mathbf{X} - \mathbf{X}_1\|_F = \sigma_2 = \sqrt{11 - \sqrt{31}} \approx 2.3307$$

3. Choosing k for an α -fraction of variance

Let $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Define

$$\text{EVR}(k) = \frac{\sum_{j=1}^k \sigma_j^2}{\sum_{j=1}^r \sigma_j^2}.$$

Explained variance ratio measures the relative amount of variance that is explained by each principal component. Pick the smallest k with $\text{EVR}(k) \geq \alpha$.

4. Principal components and intrinsic dimension

If only the first k PCs are needed to reach, $\alpha\%$ EVR, the data effectively lie on (or near) a k -dimensional manifold. That k is an empirical estimate of the *intrinsic dimension*.